

FitTrack

Plataforma nativa de salud y entrenamiento

Free · Premium · Hardware

App nativa iOS (SwiftUI) + Android (Jetpack Compose) · Freemium

Vision general

FitTrack se lanza como una **aplicacion 100 % nativa** para iOS y Android, con tres niveles de producto que crecen de forma incremental. El hardware (ESP32) se introduce solo en la tercera capa, cuando la app ya tiene una base de usuarios solida.

Plan	Precio	Descripcion
Free	Gratis	Registro de entrenamientos, objetivos basicos, estadisticas simples y ranking de comunidad con limites.
Premium	Suscripcion	Analisis avanzado, IA de recomendaciones, objetivos personalizados ilimitados, integracion con salud del SO y sin anuncios.
Hardware	Premium + dispositivo	Todo lo anterior mas monitoreo en tiempo real via ESP32: constantes vitales, calidad del aire, temperatura corporal y ambiente.

Por que apps nativas

Para una app de salud y fitness la experiencia nativa no es opcional: acceso a HealthKit (iOS) y Health Connect (Android), animaciones fluidas a 120 fps, sensores del dispositivo, notificaciones push de alta prioridad y widgets en pantalla de inicio son features que los frameworks cross-platform no replican con la misma fidelidad.

iOS	Android
Swift + SwiftUI	Kotlin + Jetpack Compose
HealthKit, CoreMotion, CoreBluetooth	Health Connect, Sensor API, Bluetooth LE
Xcode · TestFlight · App Store	Android Studio · Firebase App Distribution

El codigo compartido (logica de negocio, modelos, llamadas a la API) se extrae a una capa comun: en iOS mediante Swift Packages y en Android mediante modulos Kotlin. Esto maximiza la reutilizacion sin sacrificar la experiencia nativa.

Fases del proyecto

Fase 1 Diseño y arquitectura base

UML · esquema de datos · API contract · CI/CD inicial

Antes de escribir una sola pantalla se define la arquitectura completa. Esta fase produce los artefactos que guiarán todo el desarrollo y evita decisiones costosas de refactorizar más adelante.

Entregables:

- Diagrama de arquitectura general (C4 model: contexto, contenedores, componentes)
- Esquema de base de datos: PostgreSQL para datos de usuario y InfluxDB para series temporales
- API contract en OpenAPI 3.0: todos los endpoints documentados antes de implementar
- Estructura de repositorios GitHub: monorepo con workspaces para iOS, Android y backend
- Docker Compose inicial: postgres + influxdb + redis para entorno de desarrollo local
- Pipeline CI/CD base con GitHub Actions: lint, tests unitarios y build

GitHub en esta fase:

Se define la estrategia de ramas: **main** (producción), **develop** (integración), **feature/*** (desarrollo). Las GitHub Actions verifican que ningún merge a develop rompa el build. Los secretos (credenciales de BD, claves) se guardan como GitHub Secrets y nunca tocan el repositorio.

OpenAPI 3.0

C4 Model

GitHub Actions

Docker Compose

PostgreSQL

InfluxDB

Fase 2 Backend y API REST

Node.js · Express · JWT · WebSockets

Se implementa el servidor que sirve tanto a la app iOS como a Android. Se sigue arquitectura en capas: **Routes** → **Controllers** → **Services** → **Repository**, de modo que la lógica de negocio no depende del framework HTTP y es testeable de forma aislada.

Modulos principales:

- Auth: registro, login, refresh token, OAuth con Apple Sign-In y Google
- Usuarios: perfil, configuración, imagen de avatar
- Entrenamientos: CRUD de sesiones, ejercicios, series y repeticiones
- Objetivos: definición, progreso, completitud y logros (badges)
- Comunidad: ranking, feed público de mejoras compartidas, seguidores
- Notificaciones: push via APNs (iOS) y FCM (Android) con Firebase

Base de datos:

- PostgreSQL: usuarios, entrenamientos, objetivos, relaciones sociales
- InfluxDB: métricas de series temporales (frecuencia cardíaca, pasos, calorías)
- Redis: cache de sesiones y colas de notificaciones

Docker en esta fase:

Tres entornos definidos en docker-compose: **dev** (hot-reload, datos de seed), **staging** (imagen de producción con datos anonimizados) y **prod** (secrets inyectados en tiempo de ejecución, sin volúmenes de debug).

Node.js

Express

JWT

PostgreSQL

InfluxDB

Redis

Firebase

Docker

Fase 3 App nativa — Plan Free

SwiftUI · Jetpack Compose · HealthKit · Health Connect

Primera version publica de la app. Implementa todas las features del plan gratuito con la calidad nativa completa: animaciones del sistema, accesibilidad, modo oscuro, widgets y soporte de teclados externos.

Features del plan Free:

- Registro e inicio de sesion (email + Apple/Google Sign-In)
- Registro manual de entrenamientos: tipo, duracion, ejercicios, notas
- Seguimiento de objetivos basicos: hasta 3 objetivos activos simultaneos
- Estadisticas simples: pasos diarios, calorías estimadas, minutos activos
- Integracion con HealthKit (iOS) y Health Connect (Android) para leer pasos y frecuencia cardiaca del propio dispositivo
- Comunidad basica: ranking semanal de los 50 primeros, compartir un logro al mes
- Historial de los ultimos 30 días

Arquitectura de la app:

- iOS: MVVM con SwiftUI, Combine para reactividad, Swift Concurrency (async/await)
- Android: MVVM con Jetpack Compose, StateFlow, Coroutines
- Capa de red compartida en logica: Alamofire (iOS) / Retrofit (Android)
- Persistencia local: Core Data (iOS) / Room (Android) para funcionamiento offline

SwiftUI	Jetpack Compose	HealthKit	Health Connect	MVVM	Offline-first
---------	--------------------	-----------	----------------	------	---------------

Fase 4 Plan Premium

StoreKit 2 · Play Billing · IA de recomendaciones · Widgets

Capa de suscripcion sobre la app ya publicada. Las features premium se desbloquean en el mismo binario: no hay segunda app que descargar. La monetizacion usa las APIs nativas de cada plataforma para la maxima confianza del usuario.

Features adicionales en Premium:

- Objetivos ilimitados con objetivos personalizados y fechas limite
- Analisis avanzado: graficas de progresion, records personales, tendencias semanales y mensuales
- Recomendaciones de entrenamiento basadas en historial (modelo ligero on-device con Core ML / ML Kit)
- Integracion profunda con Apple Watch y Wear OS para capturar entrenamientos automaticamente
- Historial completo sin limite de tiempo
- Comunidad sin limites: ranking global, feed ilimitado, insignias exclusivas Premium
- Widgets de pantalla de inicio: objetivo del dia, racha actual, siguiente entrenamiento
- Exportacion de datos en CSV y PDF
- Sin publicidad

Monetizacion nativa:

- iOS: StoreKit 2 con suscripcion mensual y anual, oferta de prueba gratuita 14 dias
- Android: Google Play Billing Library 6, mismas condiciones
- El backend valida los recibos de compra con los servidores de Apple y Google para evitar fraude

Fase 5 Plan Hardware

ESP32 · BLE · MQTT · Sensores vitales

El ESP32 se integra como un periférico Bluetooth del móvil. La app nativa gestiona el emparejamiento, la recepción de datos en tiempo real y la sincronización con el backend. El plan Hardware incluye el Premium completo más las funciones del dispositivo.

Sensores del dispositivo:

- Frecuencia cardíaca y SpO2 — MAX30102
- Temperatura corporal — MLX90614 (infrarrojo)
- Acelerómetro y giroscopio — MPU-6050 (tipo de ejercicio, pasos precisos)
- Temperatura y humedad ambiente — DHT22
- Calidad del aire — CCS811 (CO2 y VOC)

Comunicación:

- BLE (Bluetooth Low Energy): conexión directa móvil-dispositivo para datos en tiempo real
- Wi-Fi + MQTT: cuando el dispositivo está en casa, envía datos directamente al backend sin pasar por el móvil

Features exclusivas Hardware:

- Dashboard en tiempo real: frecuencia cardíaca, SpO2 y temperatura durante el entrenamiento
- Alertas de zona de frecuencia cardíaca: notificación nativa cuando sales de tu zona objetivo
- Correlación ambiente-rendimiento: el backend cruza calidad del aire y temperatura con el rendimiento del entrenamiento
- Mapa de calor de recuperación: tiempo real con datos de temperatura corporal post-ejercicio

Firmware:

- C++ sobre ESP-IDF con FreeRTOS para tareas concurrentes
- OTA (Over-The-Air) updates: el firmware se actualiza desde la app sin cables

ESP32

BLE

MQTT

CoreBluetooth

Android BLE
API

OTA

FreeRTOS

Fase 6 Web pública

Next.js · SSR · SEO · App Store links

Sitio web de presentación del producto. No es un portal de usuario (ese acceso es solo desde la app nativa), sino la cara pública del proyecto para captación, información y descarga.

Secciones:

- Landing con capturas reales de la app y video demo
- Comparativa de planes Free / Premium / Hardware
- Comunidad: mejoras y logros que los usuarios hayan decidido hacer públicos
- Blog de salud y entrenamiento para SEO orgánico
- Página de descarga con smart banner que detecta iOS o Android y redirige a la tienda correcta

Tecnología:

- Next.js con App Router: SSR para la comunidad (contenido dinámico), SSG para el blog (SEO óptimo)

- Tailwind CSS para el diseño, desplegada en Vercel con CI/CD automatico desde GitHub

Next.js

SSR / SSG

Tailwind CSS

Vercel

SEO

DevOps: GitHub y Docker en detalle

Todo el proyecto vive en un **monorepo** estructurado por workspaces. Cada directorio tiene su propio Dockerfile y su pipeline de CI/CD independiente, pero comparten la misma estrategia de ramas y los mismos secretos de GitHub.

Estructura del monorepo:

```
fittrack/ apps/ios/ (Swift Package, Xcode project) apps/android/ (Gradle, Kotlin)
apps/web/ (Next.js) backend/api/ (Node.js + Express) backend/firmware/ (ESP32, C++ /
ESP-IDF) infra/ (docker-compose, Terraform, GitHub Actions) docs/ (OpenAPI spec, ADRs,
diagramas)
```

Estrategia de ramas:

```
feature/* --> develop (PR obligatoria + 1 review + CI verde) develop --> main (deploy
automatico a staging) main --> tag vX (deploy automatico a produccion)
```

Entornos Docker:

Entorno	Como se activa	Caracteristicas
development	docker compose up (local)	Hot-reload, BD con datos de seed, logs verbosos
staging	Push a develop	Imagen de produccion, datos anonimizados, TestFlight / Firebase App Distribution
production	Tag vX.Y.Z en main	Imagen optimizada, secrets de GitHub, sin volumenes de debug, alertas de Sentry

Resumen de tecnologias por capa

Capa	Lenguaje / Framework	Librerias / Herramientas clave
iOS App	Swift + SwiftUI	Combine, CoreData, HealthKit, CoreBluetooth, StoreKit 2, Core ML, WidgetKit
Android App	Kotlin + Jetpack Compose	Coroutines, Room, Health Connect, BLE API, Play Billing 6, ML Kit, Glance Widgets
Backend	Node.js + Express	TypeScript, Prisma ORM, Passport.js, Socket.io, Jest, Swagger
Base de datos	PostgreSQL + InfluxDB + Redis	Prisma Migrate, TimescaleDB (alternativa a Influx), Bull para colas
Firmware	C++ / ESP-IDF	FreeRTOS, Arduino framework, MQTT (PubSubClient), BLE stack, OTA updates
Web publica	Next.js + TypeScript	Tailwind CSS, Vercel, next-seo, App Router, MDX para blog
DevOps	GitHub Actions + Docker	Docker Compose, Terraform (infra), Sentry (errores), Railway / Fly.io (hosting backend)

El proyecto esta disenado para lanzar valor desde el primer dia: la version Free es un producto completo, Premium una mejora sustancial, y el Hardware una propuesta diferenciadora que justifica el ecosistema completo. La arquitectura nativa garantiza la mejor integracion posible con los sensores del propio movil antes de necesitar hardware externo.